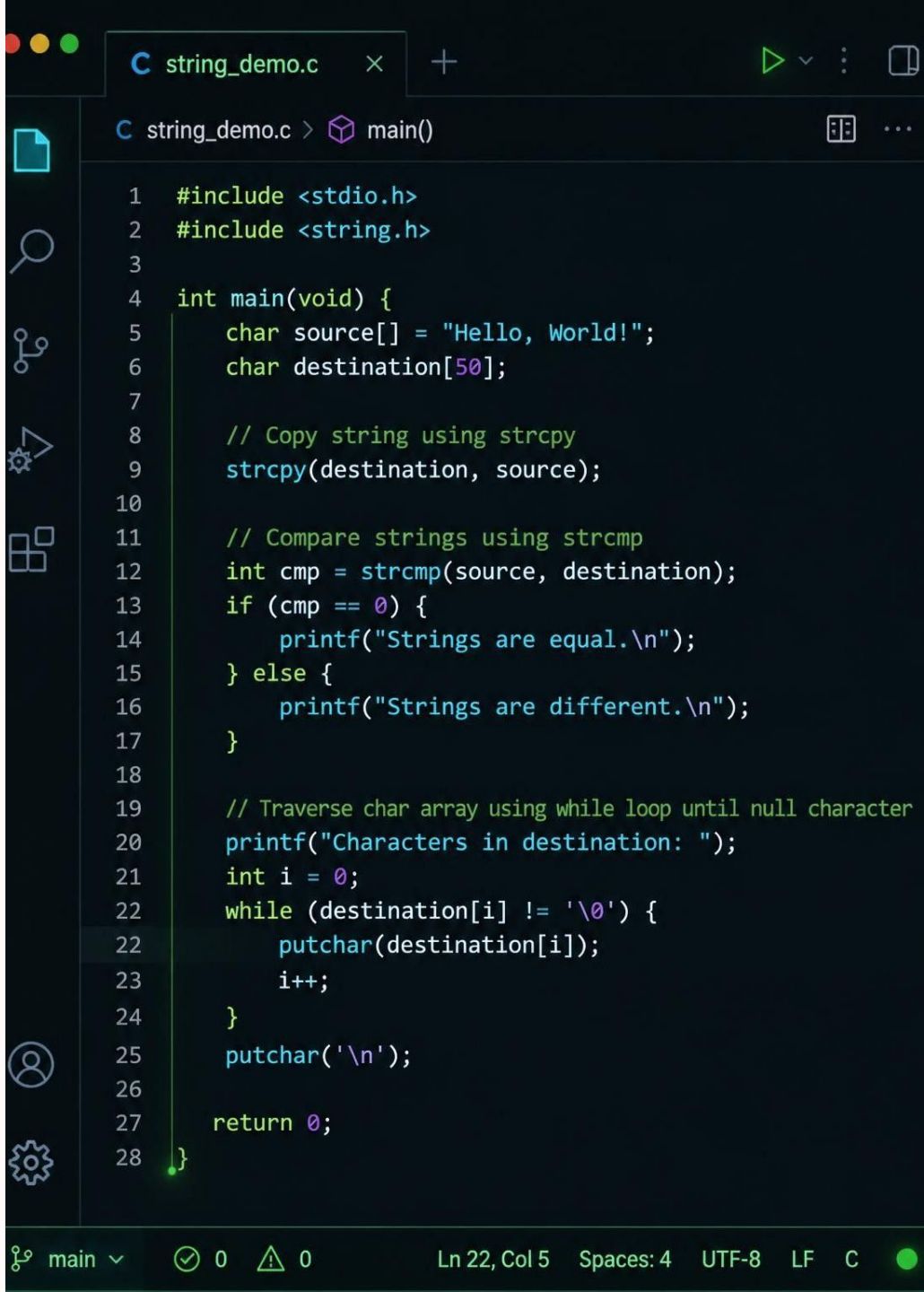
# Le Funzioni sulle Stringhe in C

Funzioni scritte a mano, libreria string.h, ricerca, caratteri e conversioni, array di stringhe

LINGUAGGIO C | STRINGHE | STRING.H



# Una Stringa come Parametro

Una stringa si passa a una funzione come qualsiasi array di char. La funzione non riceve la dimensione dell'array: la fine del testo si riconosce dal terminatore '\0'.

| Parametro | Significato |
|---|---|
| `const char s[]` | la funzione legge la stringa ma non la modifica |
| `char dest[]` | la funzione scrive nella stringa del chiamante |

Dentro la funzione sizeof(s) non dà la dimensione della stringa: s è solo l'indirizzo del primo carattere.

```
int lunghezza(const char s[]);
void copia(char dest[], const char sorg[]);


i = 0;
while (s[i] != '\0') {
    /* elabora s[i] */
    i++;
}
```

# Funzioni Scritte a Mano: Lunghezza e Copia

**Il problema:** prima di usare la libreria bisogna capire che cosa fanno davvero queste operazioni e perché hanno dei limiti.

- La lunghezza non conta il '\0'
- La copia deve scrivere anche il '\0', altrimenti dest non è una stringa
- dest deve avere almeno lunghezza(sorg) + 1 celle

copia_limitata(dest, sorg, dim) copia al massimo dim - 1 caratteri e termina sempre con '\0': il testo viene troncato ma resta valido.

```
int lunghezza(const char s[])
{
    int n = 0;
    while (s[n] != '\0') {
        n++;
    }
    return n;
}

void copia(char dest[], const char sorg[])
{
    int i = 0;
    while (sorg[i] != '\0') {
        dest[i] = sorg[i];
        i++;
    }
    dest[i] = '\0';      /* il terminatore va copiato */
}
```

# Concatenazione e Confronto

**Concatenare:** si trova il '\0' di dest e da lì si copia sorg.

```
fine = lunghezza(dest);
while (sorg[i] != '\0') {
    dest[fine + i] = sorg[i];
    i++;
}
dest[fine + i] = '\0';
```

a == b confronta gli indirizzi dei due array, non il testo: vale 0 anche quando il testo è lo stesso.

**Confrontare:** si avanza finché i caratteri sono uguali; il risultato è la differenza dei primi caratteri diversi.

```
while (a[i] == b[i] && a[i] != '\0') {
    i++;
}
return a[i] - b[i];    /* < 0, 0, > 0 */
```

| Confronto | Risultato |
|---|---|
| "Anna" e "Bruno" | negativo: Anna viene prima |
| "Mar" e "Mario" | negativo: '\0' vale 0 |
| "Zeta" e "alfa" | negativo: 'Z' = 90 < 'a' = 97 |

# La Libreria string.h

Le stesse operazioni esistono già nella libreria standard: basta includere l'header.

```
#include <string.h>

n = strlen(nome);
strcpy(copia, nome);
strcat(saluto, nome);
if (strcmp(a, b) == 0) { ... }
```

strlen restituisce size_t: si stampa con %zu oppure si converte con (int) strlen(s). strlen conta i caratteri, sizeof conta le celle dell'array.

| Funzione | Che cosa fa |
|---|---|
| `strlen(s)` | lunghezza, '\0' escluso (tipo size_t) |
| `strcpy(dest, sorg)` | copia |
| `strcat(dest, sorg)` | aggiunge in coda |
| `strcmp(a, b)` | confronto alfabetico |
| `strncpy, strncat, strncmp` | versioni con limite n |

# Le Versioni con il Limite n

**Il problema:** strcpy e strcat non conoscono la dimensione di dest. Se il testo non entra scrivono oltre la fine dell'array e sovrascrivono altre variabili.

| Funzione | Significato di n | Aggiunge il '\0'? |
|---|---|---|
| strncpy | massimo numero di caratteri copiati | no, se sorg è lunga |
| strncat | massimo numero di caratteri aggiunti | sempre |
| strncmp | numero di caratteri confrontati | — |

Con strncat il limite non è la dimensione di dest ma lo spazio ancora libero.

```
strncpy(dest, sorg, sizeof(dest) - 1);
dest[sizeof(dest) - 1] = '\0';          /* obbligatorio */

spazio_libero = sizeof(dest) - strlen(dest) - 1;
strncat(dest, sorg, spazio_libero);
```

# strcmp: Conta Solo il Segno

Lo standard garantisce solo il segno del risultato di strcmp, non il valore: il risultato si confronta sempre con 0.

- strncmp(s, prefisso, strlen(prefisso)) == 0 verifica se s inizia con il prefisso
- if (strcmp(a, b)) è vero quando le stringhe sono DIVERSE
- Il C standard non ha un confronto che ignori le maiuscole: si confrontano copie in minuscolo

Esempio: per riconoscere un cellulare italiano si sceglie il prefisso più lungo tra quelli che corrispondono: "+393" vince su "+39".

```
if (strcmp(a, b) == 0)       /* uguali */
if (strcmp(a, b) < 0)        /* a viene prima */
if (strcmp(a, b) == -1)      /* SBAGLIATO */

if (strncmp(numero, "+39", 3) == 0)   /* inizia con +39 */
```

# Cercare un Carattere o una Parola

Le tre funzioni restituiscono un puntatore, argomento del capitolo 05: per ora il risultato si usa solo per sapere se l'occorrenza esiste.

```
if (strchr(email, ' ') != NULL)      /* contiene spazi */
if (strstr(testo, "USB") == NULL)    /* non contiene USB */
```

strstr cerca una sequenza di caratteri, non una parola intera: "il" si trova anche in "invisibili". La ricerca distingue le maiuscole.

| Funzione | Cerca | Se non trova |
|---|---|---|
| `strchr(s, c)` | la prima occorrenza del carattere c | NULL |
| `strrchr(s, c)` | l'ultima occorrenza del carattere c | NULL |
| `strstr(s, parola)` | la prima occorrenza della sottostringa | NULL |

# La Posizione: Funzioni che Restituiscono l'Indice

**Il problema:** per validare un'email o separare nome ed estensione di un file non basta sapere se un carattere c'è: serve sapere dove si trova.

**La soluzione:** funzioni scritte a mano che restituiscono l'indice, oppure -1.

| Funzione | Restituisce |
|---|---|
| `prima_posizione(s, c)` | indice della prima occorrenza |
| `ultima_posizione(s, c)` | indice dell'ultima occorrenza |
| `copia_parte(dest, s, inizio, n)` | estrae n caratteri da inizio |

Per contare tutte le occorrenze si riparte da posizione + 1 finché cerca_da restituisce -1.

```
int cerca_da(const char s[], const char parola[], int da)
{
    int i = da;
    int posizione = -1;
    while (s[i] != '\0' && posizione == -1) {
        if (combacia(s, parola, i)) {
            posizione = i;
        }
        i++;
    }
    return posizione;
}
```

# Classificare i Caratteri: ctype.h

Invece di confrontare i codici ASCII a mano si usano le funzioni di ctype.h, che restituiscono un valore diverso da 0 (vero) oppure 0.

```
if (isdigit((unsigned char) s[i])) {
    cifre++;
}
s[i] = (char) toupper((unsigned char) s[i]);
```

Si scrive if (isdigit(c)), mai if (isdigit(c) == 1). Il cast (unsigned char) evita problemi con le lettere accentate.

| Funzione | Vero se c è… |
|---|---|
| isalpha(c) | una lettera |
| isdigit(c) | una cifra |
| isalnum(c) | lettera o cifra |
| isupper(c) / islower(c) | maiuscola / minuscola |
| isspace(c) | spazio, tabulazione, a capo |
| ispunct(c) | punteggiatura |

# Da Testo a Numero e Viceversa

La stringa "42" non è il numero 42: sono i caratteri '4' e '2'.

```
letti = sscanf(riga, "%d/%d/%d%c", &g, &m, &a, &resto);
/* 3 = data corretta, 4 = testo in più, < 3 = formato errato */

snprintf(riga, sizeof(riga), "%-18.18s %3d", nome, quantita);
```

Una cifra si converte nel suo valore con c - '0': (cf[6] - '0') * 10 + (cf[7] - '0').

| Funzione | Esempio | Attenzione |
|---|---|---|
| atoi(s) | "42" → 42 | "abc" → 0, "12abc" → 12: nessun errore |
| atof(s) | "3.75" → 3.75 | come atoi |
| sscanf(s, formato, …) | legge da una stringa | restituisce quanti valori ha convertito |
| snprintf(dest, dim, formato, …) | scrive in una stringa | non supera mai dim celle |

# Array di Stringhe

Un elenco di parole si memorizza in una matrice di char: ogni riga è una stringa.

```
char materie[5][16] = {
    "Italiano", "Matematica", "Informatica", "Inglese",
"Storia"
};

strcpy(elenco[n], nuovo);     /* assegnazione */
leggi_riga(citta[k], LUNG);   /* lettura */
int cerca(const char elenco[][LUNG], int n, const char
parola[]);
```

Ogni riga occupa sempre LUNG celle: LUNG deve contenere la parola più lunga più il '\0'.

| Operazione | Schema |
|---|---|
| Ricerca | strcmp(elenco[k], parola) == 0 con flag |
| Primo in ordine alfabetico | come il minimo, con strcmp < 0 |
| Scambio di due righe | tre strcpy con una stringa di appoggio |
| Aggiunta (array a tappo) | strcpy(elenco[n], nuovo); n++ |
| Cancellazione | le righe successive salgono di uno, poi n-- |

# Ignorare Maiuscole e Spazi

**Il problema:** gli utenti scrivono lo stesso dato in modi diversi: "Rossi Mario", "ROSSI MARIO", "  rossi   mario ". Con strcmp risultano tre nomi diversi.

**La soluzione:** prima del confronto si crea una copia normalizzata del testo.

- Il confronto si fa sulle copie, la stampa sugli originali
- Le posizioni trovate in una copia in minuscolo valgono anche per l'originale, perché la lunghezza è la stessa

Usato per i duplicati nelle iscrizioni, la ricerca nel catalogo, il filtro dei commenti e i comandi di magazzino.

```
void normalizza(char dest[], const char sorg[])
{
   /* minuscolo, niente spazi iniziali e finali,
      un solo spazio tra le parole */
}

normalizza(a_norm, a);
normalizza(b_norm, b);
if (strcmp(a_norm, b_norm) == 0) { ... }
```

# Errori Comuni da Evitare

**01**

Dimenticare di copiare il terminatore '\0'.

**02**

Destinazione troppo piccola per strcpy, strcat o una copia scritta a mano.

**03**

Confrontare stringhe con == invece che con strcmp.

**04**

Usare strncpy senza aggiungere il '\0' finale.

**05**

Passare a strncat la dimensione di dest invece dello spazio libero.

**06**

Confrontare il risultato di strcmp con 1 o -1.

**07**

Usare il risultato di strstr come un numero invece di confrontarlo con NULL.

**08**

Fidarsi di atoi per validare un numero: "abc" diventa 0.

# Riepilogo Concetti

| Operazione | Tecnica | Note |
|---|---|---|
| Lunghezza | strlen o ciclo fino a '\0' | '\0' escluso |
| Copia | strcpy, strncpy | spazio per il '\0' |
| Concatenazione | strcat, strncat | n = spazio libero |
| Confronto | strcmp, strncmp | solo il segno conta |
| Ricerca | strchr, strrchr, strstr | confronto con NULL |
| Posizione | funzioni con indice | -1 se non trovato |
| Tipo di carattere | isalpha, isdigit, isspace | vero = diverso da 0 |
| Maiuscole | toupper, tolower | un carattere alla volta |
| Testo → numero | sscanf | controllare quanti valori ha letto |
| Numero → testo | snprintf | mai oltre la dimensione |
| Elenchi | char elenco[N][LUNG] | assegnazione con strcpy |

# Esempi Pratici: Funzioni Scritte a Mano

Concetti base — tre file sulle operazioni fondamentali.

**1 – 010-lunghezza-e-copia.c**

**Problema:** misurare e copiare una stringa.

**Tecnica:** lunghezza, copia con il '\0', copia indipendente, copia_limitata.

**2 – 020-concatenazione.c**

**Problema:** comporre un messaggio a pezzi.

**Tecnica:** concatena dal '\0' di dest, concatena_limitata.

**3 – 030-confronto.c**

**Problema:** ordine alfabetico tra due parole.

**Tecnica:** confronta, confronto senza maiuscole, perché == non funziona.

# Esempi Pratici: Problemi con Funzioni Scritte a Mano

### 040 – Palindromo

Frasi pulite dai caratteri non alfabetici, invertite e confrontate.

### 050 – Normalizza nome

" mARIO de ROSSI " diventa "Mario De Rossi", anche con apostrofi e trattini.

### 060 – Username

Iniziale del nome + cognome, massimo 8 caratteri, cifra per gli omonimi.

### 070 – Email istituzionale

nome.cognome@dominio per studenti e docenti, con liste di distribuzione.

### 080 – Rubrica per cognome

Ricerca per inizio del cognome senza distinguere le maiuscole.

SEZIONE 2

# Esempi Pratici: Libreria string.h

Concetti base — tre file sulle funzioni della libreria.

**1 – 010-strlen-e-strcpy.c**

**Problema:** lunghezza e copia con la libreria.

**Tecnica:** strlen e sizeof, strcpy, strncpy e il '\0' da aggiungere.

**2 – 020-strcat-e-strncat.c**

**Problema:** concatenare senza superare l'array.

**Tecnica:** strcat, strncat con il calcolo dello spazio libero.

**3 – 030-strcmp-e-strncmp.c**

**Problema:** confrontare codici di prenotazione.

**Tecnica:** uguaglianza, ordine alfabetico, prefisso con strncmp.

# Esempi Pratici: Problemi con string.h

### 040 – Indirizzo postale

Etichetta di spedizione con validazione di CAP e provincia e righe troncate.

### 050 – Cambio password

Verifica della password attuale, requisiti della nuova, conferma.

### 060 – Login

Utenti e password in array paralleli, blocco dopo tre errori.

### 070 – Prefissi telefonici

Costo delle chiamate per zona, scegliendo il prefisso più lungo.

### 080 – Elezione rappresentanti

Spoglio delle schede: valide, bianche, nulle, eletti e ballottaggio.

# Esempi Pratici: Ricerca nelle Stringhe

Concetti base — tre file su caratteri e parole.

### 1 – 010-strchr-e-strrchr.c

**Problema:** trovare i separatori di un codice.

**Tecnica:** presenza con strchr, prima e ultima posizione, estrazione dei pezzi.

### 2 – 020-strstr.c

**Problema:** parole chiave nelle descrizioni.

**Tecnica:** strstr per la presenza, cerca_da per la posizione.

### 3 – 030-conta-occorrenze.c

**Problema:** quante volte compare una parola.

**Tecnica:** occorrenze sovrapposte e separate, parole intere.

# Esempi Pratici: Problemi di Ricerca

### 040 – Validazione email

Cinque regole: spazi, una sola @, nome, punto nel dominio e sua posizione.

### 050 – Filtro commenti

Parole vietate sostituite con asterischi, senza distinguere maiuscole.

### 060 – Catalogo biblioteca

Ricerca per parola chiave nei titoli e negli autori.

### 070 – Nome ed estensione

Cartella, nome ed estensione dei file e conteggio per tipo.

### 080 – Smistamento messaggi

Richieste inoltrate all'ufficio con più parole chiave.

# Esempi Pratici: Caratteri e Conversioni

Concetti base — tre file su ctype.h e conversioni.

**1 – 010-classificazione-caratteri.c**

**Problema:** di che tipo è ogni carattere.

**Tecnica:** isalpha, isdigit, isupper, isspace, ispunct.

**2 – 020-maiuscole-e-minuscole.c**

**Problema:** uniformare i dati di un'anagrafica.

**Tecnica:** toupper, tolower, iniziali maiuscole, confronto senza maiuscole.

**3 – 030-conversioni-testo-numero.c**

**Problema:** passare da testo a numero e viceversa.

**Tecnica:** atoi, atof, controllo con sscanf, snprintf e troncamento.

SEZIONE 4

# Esempi Pratici: Problemi su Caratteri e Conversioni

### 040 – Codice fiscale

Formato controllato posizione per posizione, sesso e data di nascita.

### 050 – Data di nascita

gg/mm/aaaa letta con sscanf, anni bisestili, età compiuta.

### 060 – Scontrino

Righe composte con snprintf, importi in centesimi, sconto.

### 070 – Comandi di magazzino

"CARICO viti 50": comandi testuali scomposti con sscanf.

### 080 – Cifrario di Vigenère

Cifratura con parola chiave e verifica con la decifratura.

# Esempi Pratici: Array di Stringhe

Concetti base — tre file sugli elenchi di parole.

**1 – 010-elenco-di-parole.c**

**Problema:** memorizzare un elenco.

**Tecnica:** inizializzazione, lettura con fgets, memoria occupata.

**2 – 020-ricerca-nell-elenco.c**

**Problema:** trovare un nome.

**Tecnica:** ricerca esatta e senza maiuscole, conteggio per iniziale.

**3 – 030-primo-ultimo-e-scambio.c**

**Problema:** ordine alfabetico in un elenco.

**Tecnica:** primo e ultimo con strcmp, scambio con tre strcpy.

# Esempi Pratici: Problemi con Array di Stringhe

### 040 – Dizionario

Traduzione italiano-inglese nei due sensi, con suggerimenti.

### 050 – Iscrizioni duplicate

Nomi normalizzati, duplicati segnalati, elenco pulito.

### 060 – Menu ristorante

Menu per categoria, ordinazione e conto con coperto.

### 070 – Rubrica modificabile

Array a tappo: aggiunta, ricerca, cancellazione.

### 080 – Anagrammi

Conteggio delle lettere per riconoscere gli anagrammi.